

Competitive Programming Notebook

py.Sandu

Contents

1	Graph	2
1.1	Dfs	2
1.2	Eulerean Path	2
1.3	Ford-fulkerson	2
1.4	Mst Prim	2
1.5	Dijkstra	3
1.6	Floyd-warshall	3
1.7	Mst Kruskal	3
1.8	Bellman-ford	3
1.9	Bfs	4
2	Geometry	4
2.1	Winding Number	4
2.2	Isleft	4
2.3	Point	4
2.4	Convex Hull	4
2.5	Cross Product	5
3	String	5
3.1	Kmp	5
4	General	5
5	DP	5
5.1	Coins Change	5
5.2	Kadane Algorithm	5
5.3	Knapsack	6
5.4	Longest Increasing Subsequence	6
6	Math	6
6.1	Matrix Multiplication	6
6.2	Modular Exponentiation	6
6.3	Solve Linear System	6
6.4	Matrix Fibonacci	7
6.5	Factorial	7
6.6	Siege Of Eratosthenes	7
6.7	Extended Euclidean Algorithm	7
7	DS	8
7.1	Monotonic Stack	8
7.2	Bit	8
7.3	Bit2d	8
7.4	Dsu	9
7.5	Sparsetable	9
7.6	Trie	9
7.7	Psum2d	10

1 Graph

1.1 Dfs

```

1 // Depth First Search (DFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 // Recursive DFS
6 void dfs(const Graph& graph, int vertex, vector<bool>
    & visited) {
7     visited[vertex] = true;
8
9     for (int neighbor : graph[vertex]) {
10         if (!visited[neighbor]) {
11             dfs(graph, neighbor, visited);
12         }
13     }
14 }
15
16 // Iterative DFS
17 void dfsIterative(const Graph& graph, int start) {
18     int V = graph.size();
19     vector<bool> visited(V, false);
20     stack<int> s;
21     s.push(start);
22
23     while (!s.empty()) {
24         int vertex = s.top();
25         s.pop();
26
27         if (!visited[vertex]) {
28             visited[vertex] = true;
29
30             for (int neighbor : graph[vertex]) {
31                 if (!visited[neighbor]) {
32                     s.push(neighbor);
33                 }
34             }
35         }
36     }

```

1.2 Eulerian Path

```

1 // Find the Eulerian path in a graph using Hierholzer
  's algorithm
2 // Time complexity:  $O(E)$  where E is the number of
  edges in the graph
3 // Use cases: visit every edge once, DNA sequencing,
  puzzles, drawing figures
4
5 vector<int> findEulerianPath(vector<vector<int>>& adj
    , int start) {
6
7     vector<int> path;
8     stack<int> s;
9     s.push(start);
10
11     while (!s.empty()) {
12         int v = s.top();
13         if (adj[v].empty()) {
14             path.push_back(v);
15             s.pop();
16         } else {
17             int u = adj[v].back();
18             adj[v].pop_back(); // Remove the edge
19             from v to u
20             s.push(u);
21         }
22     }
23     reverse(path.begin(), path.end());

```

```

24     return path;
25 }

```

1.3 Ford-fulkerson

```

1 // Find maximum flow in graph
2 // Time complexity:  $O(|V| * E^2)$ 
3 // Space complexity:  $O(V)$ 
4
5 bool bfs(int residual[V][V], int start, int target,
    vi parent){
6     vector<bool> visited(V, false);
7
8     queue<int> q;
9     q.push(start);
10    visited[start] = true;
11    parent[start] = -1;
12
13    while(!q.empty()){
14        int u = q.front();
15        q.pop();
16
17        for(int v = 0 ; v <= V ; v++){
18            if(!visited[v] and residual[u][v] > 0){
19                if(v == target){
20                    parent[v] = u;
21                    return true;
22                }
23                q.push(v);
24                parent[v] = u;
25                visited[v] = true;
26            }
27        }
28    }
29    return false;
30 }
31
32 int fordFulkerson(int g[V][V], int start, int target)
    {
33     int u, v;
34     int residual[V][V];
35
36     for(u = 0 ; u < V ; u++){
37         for(v = 0 ; v < V ; v++){
38             residual[u][v] = g[u][v];
39         }
40     }
41     vi parent(V);
42     int maxFlow = 0;
43
44     while(bfs(residual, start, target, parent)){
45         int pathFlow = INT_MAX;
46         for(v = target ; v != start ; v = parent[v]){
47             u = parent[v];
48             pathFlow = min(pathFlow, residual[u][v]);
49         }
50         for(v = target ; v != start ; v = parent[v]){
51             u = parent[v];
52             residual[u][v] -= pathFlow;
53             residual[v][u] += pathFlow;
54         }
55         maxFlow += pathFlow;
56     }
57     return maxFlow;
58 }

```

1.4 Mst Prim

```

1 // Prim's algorithm for finding the Minimum Spanning
  Tree
2 // Time complexity:  $O(E \log V)$  where E is the number
  of edges and V is the number of vertices

```

```

3
4 struct Edge {
5     int u, v, weight;
6     bool operator<(const Edge& other) const {
7         return weight > other.weight; // Min-heap
8     }
9 };
10
11 vector<Edge> primMST(const vector<vector<int>>& graph
12 ) {
13     int n = graph.size();
14     vector<bool> visited(n, false);
15     vector<Edge> mst;
16     priority_queue<Edge> pq;
17
18     // Start with the first vertex
19     visited[0] = true;
20     for (int i = 0; i < n; ++i) {
21         if (graph[0][i] != 0) {
22             pq.push({0, i, graph[0][i]});
23         }
24     }
25
26     while (!pq.empty()) {
27         Edge current = pq.top();
28         pq.pop();
29
30         if (visited[current.v]) continue;
31
32         visited[current.v] = true;
33         mst.push_back(current);
34
35         for (int i = 0; i < n; ++i) {
36             if (!visited[i] && graph[current.v][i] !=
37                 0) {
38                 pq.push({current.v, i, graph[current.
39 v][i]});
40             }
41         }
42     }
43
44     return mst;
45 }

```

1.5 Dijkstra

```

1 // Find shortest path from a source vertex to all
2 // other vertices in a graph with non-negative edge
3 // weights
4 // Time Complexity:  $O((V + E) \log V)$ 
5 // Space Complexity:  $O(V)$ 
6 // Use cases: shortest path in non-negative weighted
7 // graphs
8
9 void dijkstra(vector<vector<pair<int, int>>>& adj,
10 int src) {
11
12     int n = adj.size() - 1;
13     vector<int> dist(n+1, INT_MAX);
14     dist[src] = 0;
15
16     priority_queue<pair<int, int>, vector<pair<int,
17 int>>, greater<pair<int, int>>> pq; // Min-heap
18     priority_queue
19     pq.push({0, src});
20
21     while (!pq.empty()) {
22
23         auto [d, u] = pq.top();
24         pq.pop();
25
26         if (d > dist[u]) continue;
27
28

```

```

22         for (auto p : adj[u]) {
23
24             int v = p.first;
25             int weight = p.second;
26
27             if (dist[u] != INT_MAX && dist[u] +
28                 weight < dist[v]) {
29                 dist[v] = dist[u] + weight;
30                 pq.push({dist[v], v});
31             }
32         }
33     }

```

1.6 Floyd-warshall

```

1 // Find the shortest paths in a weighted graph with
2 // positive or negative edge weights (but with no
3 // negative cycles).
4 // Time Complexity:  $O(V^3)$ 
5 // Space Complexity:  $O(V^2)$ 
6 // Use cases: reachability, minimax distance, global
7 // negative cycle detection( $dist[i][i] < 0$ )
8
9 void floydWarshall(vector<vector<int>>& dist) { //
10     dist(INF)
11
12     int n = dist.size() - 1;
13     for(int i = 1; i <= n; i++) dist[i][i] = 0;
14
15     for (int k = 1; k <= n; k++) {
16         for (int i = 1; i <= n; i++) {
17             for (int j = 1; j <= n; j++) {
18                 if (dist[i][k] != INF and dist[k][j]
19                     != INF) {
20                     dist[i][j] = min(dist[i][j], dist
21 [i][k] + dist[k][j]);
22                 }
23             }
24         }
25     }
26 }

```

1.7 Mst Kruskal

```

1 // Kruskal algorithm for finding the Minimum Spanning
2 // Tree
3 // Time Complexity:  $O(E \log E)$ 
4 // Space complexity:  $O(V + E)$ 
5 // Use cases: connect nodes with min/max cost,
6 // clustering, minimax path
7
8 // edges: {weight, u, v}
9 pair<int, vector<vector<int>>> kruskalMST(vector<
10 array<int, 3>>& edges, int n){
11     int m = (int)edges.size();
12     sort(edges.begin(), edges.end());
13     vector<vector<int>> mst(n+1);
14     DSU dsu(n+1); int cost = 0;
15     for(int i=0; i<m; i++){
16         int a = edges[i][1], b = edges[i][2];
17         if(dsu.merge(a, b)){
18             mst[a].push_back(b);
19             mst[b].push_back(a);
20             cost += edges[i][0];
21         }
22     }
23     return {cost, mst};
24 }

```

1.8 Bellman-ford

```

1 // Find the shortest path from a source vertex to all
  other vertices in a graph with negative weight
  edges and detect negative weight cycles.
2 // Time Complexity: O(V * E)
3 // Space Complexity: O(V)
4 // Use cases: SSSP with negative edges, negative
  cycle detection, system of difference constraints
  , shortest path with at most K edges
5
6 void bellmanFord(vector<vector<pair<int, int>>>& adj,
  int src) {
7
8   int n = adj.size() - 1;
9   vector<int> dist(n+1, INF);
10  dist[src] = 0;
11
12  // Relax all edges n-1 times
13  for (int i = 1; i < n; i++) {
14    for (int u = 1; u <= n; u++) {
15      for (auto& [v, w] : adj[u]) {
16        if (dist[u] != INF and dist[u] + w <
17          dist[v]) {
18          dist[v] = dist[u] + w;
19        }
20      }
21    }
22
23    // Check for negative weight cycles
24    for (int u = 1; u <= n; u++) {
25      for (auto& [v, w] : adj[u]) {
26        if (dist[u] != INF and dist[u] + w < dist
27          [v]) {
28          cout << "Graph contains negative
29          weight cycle\n";
30          return;
31        }
32      }
33    }
34  }
35 }

```

1.9 Bfs

```

1 // Breadth First Search (BFS)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4
5 void bfs(const Graph& graph, int start) {
6
7   int V = graph.size();
8   vector<bool> visited(V, false);
9   queue<int> q;
10
11  visited[start] = true;
12  q.push(start);
13
14  while (!q.empty()) {
15    int vertex = q.front();
16    q.pop();
17
18    for (int neighbor : graph[vertex]) {
19      if (!visited[neighbor]) {
20        visited[neighbor] = true;
21        q.push(neighbor);
22      }
23    }
24  }
25 }

```

2 Geometry

2.1 Winding Number

```

1 // Check if a point is inside a polygon using the
  winding number algorithm
2
3 bool isInsidePolygon(const Point& p, const vector<
  Point>& polygon) {
4   int windingNumber = 0;
5   int n = polygon.size();
6
7   for (int i = 0; i < n; i++) {
8     const Point& v1 = polygon[i];
9     const Point& v2 = polygon[(i + 1) % n];
10
11    if (v1.y <= p.y) {
12      if (v2.y > p.y && isLeft(p, v1, v2)) {
13        ++windingNumber;
14      }
15    } else {
16      if (v2.y <= p.y && !isLeft(p, v1, v2)) {
17        --windingNumber;
18      }
19    }
20  }
21
22  return windingNumber != 0;
23 }

```

2.2 Isleft

```

1 // Check if a point is to the left of a segment
  defined by two points
2 bool isLeft(const Point& p, const Point& v1, const
  Point& v2) {
3   return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
  y) * (p.x - v1.x) > 0;
4 }

```

2.3 Point

```

1 struct Point {
2   double x, y;
3 };

```

2.4 Convex Hull

```

1 // Returns a list of points on the convex hull
2 // in counter-clockwise order
3 vector<Point> convex_hull(vector<Point> A)
4 {
5   int n = A.size(), k = 0;
6
7   if (n <= 3)
8     return A;
9
10  vector<Point> ans(2 * n);
11
12  // Sort points lexicographically
13  sort(A.begin(), A.end());
14
15  // Build lower hull
16  for (int i = 0; i < n; ++i) {
17
18    // If the point at K-1 position is not a part
19    // of hull as vector from ans[k-2] to ans[k
20    -1]
21    // and ans[k-2] to A[i] has a clockwise turn
22    while (
23      k >= 2

```

```

23         && cross_product(ans[k - 2], ans[k - 1],
24         A[i])
25         <= 0)
26         k--;
27         ans[k++] = A[i];
28     }
29     // Build upper hull
30     for (size_t i = n - 1, t = k + 1; i > 0; --i) {
31         // If the point at K-1 position is not a part
32         // of hull as vector from ans[k-2] to ans[k
33         -1]
34         // and ans[k-2] to A[i] has a clockwise turn
35         while (k >= t
36         && cross_product(ans[k - 2], ans[k -
37         1],
38         A[i - 1])
39         <= 0)
40         k--;
41         ans[k++] = A[i - 1];
42     }
43     // Resize the array to desired size
44     ans.resize(k - 1);
45     return ans;
46 }
47 }

```

2.5 Cross Product

```

1 double cross_product(Point O, Point A, Point B)
2 {
3     return (A.x - O.x) * (B.y - O.y)
4         - (A.y - O.y) * (B.x - O.x);
5 }

```

3 String

3.1 Kmp

```

1 // Find all occurrences of a pattern in a given text
2 // using the Knuth-Morris-Pratt (KMP) algorithm
3 // Time complexity: O(n + m) where n is the length of
4 // the text and m is the length of the pattern
5
6 vector<int> LPS(const string& pattern) {
7     int m = pattern.size();
8     vector<int> lps(m, 0);
9     int len = 0; // length of the previous longest
10    prefix suffix
11    int i = 1;
12
13    while (i < m) {
14        if (pattern[i] == pattern[len]) {
15            len++;
16            lps[i] = len;
17            i++;
18        } else {
19            if (len != 0) {
20                len = lps[len - 1];
21            } else {
22                lps[i] = 0;
23                i++;
24            }
25        }
26    }
27    return lps;
28 }
29
30 vector<int> KMP(const string& text, const string&
31 pattern) {

```

```

32 vector<int> lps = LPS(pattern);
33 vector<int> occurrences;
34 int n = text.size();
35 int m = pattern.size();
36 int i = 0; // index for text
37 int j = 0; // index for pattern
38
39 while (i < n) {
40     if (text[i] == pattern[j]) {
41         i++;
42         j++;
43     }
44     if (j == m) {
45         occurrences.push_back(i - j); // Pattern
46         found at index (i - j)
47         j = lps[j - 1]; // Continue to search for
48         the next match
49     } else if (i < n && text[i] != pattern[j]) {
50         if (j != 0) {
51             j = lps[j - 1];
52         } else {
53             i++;
54         }
55     }
56 }
57 return occurrences;
58 }

```

4 General

5 DP

5.1 Coins Change

```

1 // Find the minimum number of coins needed to make a
2 // given amount using a set of coin denominations
3 // Time complexity: O(n * m) where n is the amount
4 // and m is the number of coin denominations
5
6 int coinChange(const vector<int>& coins, int amount)
7 {
8     vector<int> dp(amount + 1, INT_MAX);
9     dp[0] = 0;
10
11     for (int i = 1; i <= amount; i++) {
12         for (const int& coin : coins) {
13             if (coin <= i && dp[i - coin] != INT_MAX)
14             {
15                 dp[i] = min(dp[i], dp[i - coin] + 1);
16             }
17         }
18     }
19
20     return dp[amount] == INT_MAX ? -1 : dp[amount];
21 }

```

5.2 Kadane Algorithm

```

1 // Find maximum subarray sum using Kadane's algorithm
2 // Time complexity: O(n) where n is the length of the
3 // input array
4
5 int maxSubArraySum(const vector<int>& arr) {
6     int maxSum = arr[0];
7     int maxLocal = arr[0];
8
9     for (int i = 0; i < arr.size(); i++) {
10         maxLocal = max(arr[i], maxLocal + arr[i]);
11         maxSum = max(maxSum, maxLocal);
12     }
13 }

```

```

12
13     return maxSum;
14 }

```

5.3 Knapsack

```

1 // General take/not take dp problem
2 // Time Complexity: approx.  $O(n * m)$ 
3 // Space Complexity:  $O(n * m) / O(m)$ 
4 // Use cases: target sum(subset, partition), knapsack
5
6 // Top-down
7
8 const int MAXN = 1e3, MAXW = 1e3;
9
10 int knapsack_top_down(vector<int>& v, vector<int>& w,
11     int x){
12     int n = (int)v.size();
13     vector<vector<int>> dp(MAXN, vector<int>(MAXW,
14         -1));
15     auto rec = [&](auto self, int i, int m) -> int {
16         if(i >= n || m == 0) return 0;
17         if(dp[i][m] != -1) return dp[i][m];
18         int not_take = self(self, i+1, m);
19         int take = 0;
20         if(m >= w[i]){
21             take = v[i] + self(self, i+1, m - w[i]);
22         }
23         return dp[i][m] = max(take, not_take);
24     };
25
26     return rec(rec, 0, x);
27 }
28
29 // Bottom-up
30
31 int knapsack_bottom_up(vector<int>& v, vector<int>& w,
32     int x){
33     int n = (int)v.size();
34     vector<int> dp(x+1, 0);
35
36     for(int i=0; i < n; i++){
37         for(int m=x; m >= w[i]; m--){
38             dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
39         }
40     }
41
42     return dp[x];
43 }

```

5.4 Longest Increasing Subsequence

```

1 // Find the length of the longest increasing
2 // subsequence in a given array
3 // Time complexity:  $O(n \log n)$  where n is the length
4 // of the input array
5
6 int lis(vector<int>& arr) {
7     vector<int> dp;
8     for (int num : arr) {
9         auto it = lower_bound(dp.begin(), dp.end(),
10             num);
11         if (it == dp.end()) {
12             dp.push_back(num);
13         } else {
14             *it = num;
15         }
16     }
17
18     return dp.size();
19 }

```

6 Math

6.1 Matrix Multiplication

```

1 // Multiply two matrices A and B, and return the
2 // result in matrix C.
3 // Time Complexity:  $O(n^3)$ 
4 // Space Complexity:  $O(n^2)$ 
5
6 vector<vector<int>> matrixMultiplication(const vector
7     <vector<int>>& A, const vector<vector<int>>& B) {
8     int n = A.size();
9     vector<vector<int>> C(n, vector<int>(n, 0)); //
10     Initialize result matrix with zeros
11
12     for (int i = 0; i < n; i++) {
13         for (int j = 0; j < n; j++) {
14             for (int k = 0; k < n; k++) {
15                 C[i][j] += A[i][k] * B[k][j];
16             }
17         }
18     }
19
20     return C;
21 }

```

6.2 Modular Exponentiation

```

1 // Calculates  $(x^n) \% MOD$ 
2 // Time Complexity:  $O(\log n)$ 
3 // Space Complexity:  $O(1)$ 
4
5 int fexp(int x, int b, int MOD){
6     int res = 1;
7
8     while(b > 0){
9         if(b & 1) res = (res * x) % MOD;
10        x = (x * x) % MOD;
11        b >>= 1;
12    }
13
14    return res;
15 }

```

6.3 Solve Linear System

```

1 // Solve a system of linear equations using Cramer's
2 // rule
3 // Time complexity:  $O(n^4)$  in the worst case
4
5 // Function to calculate the determinant of a matrix
6 double determinant(const vector<vector<double>>&
7     matrix) {
8     int n = matrix.size();
9     if (n == 1) {
10         return matrix[0][0];
11     }
12
13     double det = 0.0;
14     for (int col = 0; col < n; col++) {
15         vector<vector<double>> submatrix(n - 1,
16             vector<double>(n - 1));
17         for (int i = 1; i < n; ++i) {
18             for (int j = 0; j < n; ++j) {
19                 if (j < col) {
20                     submatrix[i - 1][j] = matrix[i][j];
21                 } else if (j > col) {
22                     submatrix[i - 1][j - 1] = matrix[
23                         i][j];
24                 }
25             }
26         }
27
28         det += (col % 2 == 0 ? 1 : -1) * matrix[0][
29             col] * determinant(submatrix);
30     }
31 }

```

```

23     }
24     return det;
25 }
26
27 // Function to solve a system of linear equations
28 // using Cramer's rule
29 vector<double> solveLinearSystem(const vector<vector<
30 double>>& A, const vector<double>& b) // A is the
31 coefficient matrix and b is the constant vector
32 {
33     int n = A.size();
34     double detA = determinant(A);
35
36     vector<double> solution(n);
37     for (int i = 0; i < n; i++) {
38         vector<vector<double>> Ai = A;
39         for (int j = 0; j < n; ++j) {
40             Ai[j][i] = b[j];
41         }
42         solution[i] = determinant(Ai) / detA; //
43         Cramer's rule
44     }
45     return solution;
46 }

```

6.4 Matrix Fibonacci

```

1 // Calculate the Nth Fibonacci number using matrix
2 // exponentiation.
3 // Time Complexity: O(log N)
4 // Space Complexity: O(1)
5 void matrixExponentiation(vector<vector<long long>>&
6 matrix, int power) {
7     int n = matrix.size();
8     vector<vector<long long>> result(n, vector<long
9 long>(n, 0));
10
11     // Initialize result as the identity matrix
12     for (int i = 0; i < n; i++) {
13         result[i][i] = 1;
14     }
15
16     while (power) {
17         if (power % 2 == 1) {
18             matrixMultiplication(result, matrix,
19 result);
20         }
21         matrixMultiplication(matrix, matrix, matrix);
22         power /= 2;
23     }
24
25     matrix = result;
26 }

```

6.5 Factorial

```

1 // Time Complexity: O(N) for precompute, O(1) for nCr
2 // Space Complexity: O(N)
3 // Use cases: calculating combinations, permutations,
4 // probabilities
5 const int MAXN, MOD;
6 vector<int> fact(MAXN), ifact(MAXN);
7
8 void precompute(){
9
10     fact[0] = 1; ifact[0] = 1;
11     for(int i=1; i<MAXN; i++){
12         fact[i] = (i * fact[i-1]) % MOD;
13     }

```

```

14
15     ifact[MAXN-1] = fexp(fact[MAXN-1], MOD-2, MOD);
16     for(int i=MAXN-2; i >= 1; i--){
17         ifact[i] = (ifact[i+1]*(i+1)) % MOD;
18     }
19 }
20
21 int nCr(int n, int r){
22     if(r < 0 or r > n) return 0;
23     int num = fact[n];
24     int den = (ifact[r] * ifact[n - r]) % MOD;
25     return (num * den) % MOD;
26 }

```

6.6 Sieve Of Eratosthenes

```

1 // Find all prime numbers up to a given limit using
2 // the Sieve of Eratosthenes algorithm
3 // Time complexity: O(n log log n) where n is the
4 // limit
5 // This idea can also be used to construct a table
6 // for the smallest prime factors, O(log N) per
7 // query
8
9 vector<int> findPrimes(int limit) {
10     vector<bool> isPrime(limit + 1, true);
11     isPrime[0] = isPrime[1] = false;
12
13     for (int i = 2; i * i <= limit; i++) {
14         if (isPrime[i]) {
15             for (int j = i * i; j <= limit; j += i) {
16                 isPrime[j] = false;
17             }
18         }
19     }
20
21     vector<int> primes;
22     for (int i = 2; i <= limit; i++) {
23         if (isPrime[i]) {
24             primes.push_back(i);
25         }
26     }
27     return primes;
28 }
29
30 vector<int> buildSPF(int limit){
31     vector<int> spf(limit+1);
32     iota(spf.begin(), spf.end(), 0);
33     for(int i = 2; i * i <= limit; i++) {
34         if(spf[i] == i){
35             for(int j = i * i; j <= limit; j += i) {
36                 if(spf[j] == j) spf[j] = i;
37             }
38         }
39     }
40
41     return spf;
42 }
43
44 vector<int> getFactors(vector<int>& spf, int x){
45     vector<int> factors;
46     while(x > 1){
47         factors.push_back(spf[x]);
48         x /= spf[x];
49     }
50
51     return factors;
52 }

```

6.7 Extended Euclidean Algorithm

```

1 // Extended Euclidean Algorithm

```

```

2 // Returns (gcd(a, b), x, y) such that a*x + b*y =
  gcd(a, b)
3
4 int extendedEuclidean(int a, int b, int &x, int &y){
5     if(a == 0){
6         x = 0;
7         y = 1;
8         return b;
9     }
10
11     int x1, y1;
12     int gcd = extendedEuclidean(b % a, a, x1, y1);
13     x = y1;
14     y = x1 - (b / a) * y1;
15     return gcd;
16 }

```

7 DS

7.1 Monotonic Stack

```

1 // Monotonic Stack, next greater number in a circular
  array
2 // Time Complexity: O(N)
3 // Space Complexity: O(N)
4 // Use cases: ring problems, finding next larger
  obstacle in a loop
5
6 int n; vector<int> arr(n);
7 vector<int> res(n, -1);
8 stack<int> st;
9
10 for (int i = 0; i < 2 * n; i++) {
11     int num = arr[i % n];
12     while (!st.empty() and arr[st.top()] < num) {
13         // remove os menores
14         res[st.top()] = num;
15         st.pop();
16     }
17     if (i < n) {
18         st.push(i);
19     }
20 }
21 // Monotonic Stack, next smaller element in a linear
  array
22 // Use cases: largest rectangle in histogram,
  subarray contribution
23
24 for (int i = 0; i < n; i++) {
25     while (!st.empty() and arr[st.top()] > arr[i
26 ]) { // remove os maiores
27         res[st.top()] = arr[i];
28         st.pop();
29     }
30     st.push(i);
31 }

```

7.2 Bit

```

1 // Binary Indexed Tree(BIT)
2 // Time complexity: query and update log N
3 // Space complexity: O(N)
4 // Use cases: dynamic psum, inversion counting,
  dynamic multiset
5
6 struct BIT{
7
8     int n; vector<int> bit;
9     BIT(int n) : n(n){
10         bit.assign(n+1, 0);

```

```

11     }
12
13     void update(int i, int x){
14         while(i <= n){
15             bit[i] += x;
16             i += i & -i;
17         }
18     }
19
20     int query(int i){
21         int sum = 0;
22         while(i > 0){
23             sum += bit[i];
24             i -= i & -i;
25         }
26         return sum;
27     }
28
29     int queryRange(int l, int r){
30         return query(r) - query(l-1);
31     }
32
33     // retorna o índice do k-ésimo elemento da bit
34     int find_kth(int k){
35
36         int idx = 0;
37         int max_bit = __lg(n);
38
39         for(int i = max_bit; i >= 0; i--){
40             int next_idx = idx + (1ll << i);
41             if(next_idx <= n and bit[next_idx] < k){
42                 idx = next_idx;
43                 k -= bit[next_idx];
44             }
45         }
46         return idx+1;
47     }
48 };
49
50

```

7.3 Bit2d

```

1 // Binary Indexed Tree(BIT) 2D
2 // Time complexity: query and update log N * log M
3 // Space complexity: O(N*M)
4 // Use cases: dynamic 2D prefix sums, counting active
  points in a dynamic rectangle
5
6 struct BIT2D {
7
8     int n, m; vector<vector<int>> bit;
9     BIT2D(int n, int m) : n(n), m(m) {
10         bit.assign(n+1, vector<int>(m+1, 0));
11     }
12
13     void update(int x, int y, int val) {
14         for (int i = x; i <= n; i += i & -i) {
15             for (int j = y; j <= m; j += j & -j) {
16                 bit[i][j] += val;
17             }
18         }
19     }
20
21     int query(int x, int y) {
22         int sum = 0;
23         for (int i = x; i > 0; i -= i & -i) {
24             for (int j = y; j > 0; j -= j & -j) {
25                 sum += bit[i][j];
26             }
27         }
28         return sum;
29     }
30 }

```



```

30
31 int queryRange(int x1, int y1, int x2, int y2) {
32     return query(x2, y2)
33         - query(x1 - 1, y2)
34         - query(x2, y1 - 1)
35         + query(x1 - 1, y1 - 1);
36 }
37
38 };

```

7.4 Dsu

```

1 // Disjoint Set Union(DSU)
2 // Time Complexity: aprox. O(1)
3 // Space Complexity: O(n)
4 // Use cases: connected components, cycle detection,
  offline edge deletion
5
6 struct DSU{
7
8     vector<int> parent, size;
9
10    DSU(int n){
11        parent.assign(n+1, 0);
12        size.assign(n+1, 1);
13        iota(parent.begin(), parent.end(), 0);
14    }
15
16    int find(int v){
17        if(v == parent[v]) return v;
18        return parent[v] = find(parent[v]);
19    }
20
21    bool same(int a, int b){
22        return find(a) == find(b);
23    }
24
25    bool merge(int a, int b){
26        a = find(a);
27        b = find(b);
28        if(a != b){
29            if(size[a] < size[b]) swap(a, b);
30            parent[b] = a;
31            size[a] += size[b];
32            return true;
33        }
34        return false;
35    }
36
37 };

```

7.5 Sparsetable

```

1 // Sparse Table
2 // Time Complexity: O(n * log n) build, O(1) query
3 // Space complexity: O(n * log n)
4 // Idempotent operation f(a, a) = a (min, max, gcd,
  and, or)
5 // Use cases: idempotent functions range queries
6 struct SparseTable{
7
8     int n, p;
9     vector<vector<int>> st;
10
11    SparseTable(vector<int> &arr){
12        n = (int) arr.size();
13        p = __lg(n);
14        st.assign(p+1, vector<int>(n, 0));
15
16        for(int j=0; j<n; j++) st[0][j] = arr[j];
17        for(int i = 1; i <= p; i++){
18            int k = (1ll << (i-1));

```

```

19         int lim = n - 2*k + 1;
20         for(int j=0; j<lim; j++){
21             st[i][j] = op(st[i-1][j], st[i-1][j+k
22         });
23     }
24 }
25
26 int query(int l, int r){
27     int len = r - l + 1;
28     int p_local = __lg(len), k = (1ll << p_local)
29     ;
30     return op(st[p_local][l], st[p_local][r - k +
31     1]);
32 };

```

7.6 Trie

```

1 // class for a node of the trie
2 class TrieNode {
3 public:
4     bool endofWord;
5     TrieNode* children[26];
6
7     // Constructore to initialize a trie node
8     TrieNode()
9     {
10         endofWord = false;
11         for (int i = 0; i < 26; i++) {
12             children[i] = nullptr;
13         }
14     }
15 };
16
17 // class for the Trie implementation
18 class Trie {
19 private:
20     TrieNode* root;
21
22 public:
23     Trie() { root = new TrieNode(); }
24
25     // Function to insert a word into the trie
26     void insert(string word)
27     {
28         TrieNode* node = root;
29         for (char c : word) {
30             int index = c - 'a';
31             if (!node->children[index]) {
32                 node->children[index] = new TrieNode
33             };
34             node = node->children[index];
35         }
36         node->endofWord = true;
37     }
38
39     // Function to search a word in the trie
40     bool search(string word)
41     {
42         TrieNode* node = root;
43         for (char c : word) {
44             int index = c - 'a';
45             if (!node->children[index]) {
46                 return false;
47             }
48             node = node->children[index];
49         }
50         return node->endofWord;
51     }
52 }

```

```

53 // Function to check if there is any word in the
    trie
54 // that starts with the given prefix
55 bool startsWith(string prefix)
56 {
57     TrieNode* node = root;
58     for (char c : prefix) {
59         int index = c - 'a';
60         if (!node->children[index]) {
61             return false;
62         }
63         node = node->children[index];
64     }
65     return true;
66 }
67
68 // Function to delete a word from the trie
69 void deleteWord(string word)
70 {
71     TrieNode* node = root;
72     for (char c : word) {
73         int index = c - 'a';
74         if (!node->children[index]) {
75             return;
76         }
77         node = node->children[index];
78     }
79     if (node->endofWord == true) {
80         node->endofWord = false;
81     }
82 }
83
84 // Function to print the trie
85 void print(TrieNode* node, string prefix) const
86 {
87     if (node->endofWord) {
88         cout << prefix << endl;
89     }
90     for (int i = 0; i < 26; i++) {
91         if (node->children[i]) {
92             print(node->children[i],

```

```

93             prefix + char('a' + i));
94         }
95     }
96 }
97
98 // Function to start printing from the root
99 void print() const { print(root, ""); }
100 };

```

7.7 Psum2d

```

1 // Psum2D
2 // Time Complexity: Query O(1)
3 // Space Complexity: O(N*M)
4 // Use cases: static 2d range sum, fixed-size window
    search, max sum submatrix
5
6 int n, m;
7 vector<vector<int>> arr(n+1, vector<int>(m+1, 0));
8 vector<vector<int>> psum(n+1, vector<int>(m+1, 0));
9
10 for (int i = 1; i <= n; i++) {
11     for (int j = 1; j <= m; j++) {
12         cin >> arr[i][j];
13         psum[i][j] = arr[i][j]
14             + psum[i][j-1]
15             + psum[i-1][j]
16             - psum[i-1][j-1];
17     }
18 }
19
20
21 // canto superior esquerdo (x1, y1) canto inferior
    direito (x2, y2)
22 int x1, y1, x2, y2;
23 int res = psum[x2][y2]
24     - psum[x1 - 1][y2]
25     - psum[x2][y1 - 1]
26     + psum[x1 - 1][y1 - 1];

```